Wrocław University
of Science and Technology

Information Systems Modeling
Lab03 group 3

WroclArts System
– Documentation –

— Data Storage Structure —

Student

Maxime Luce
Student ID: 294620

Teacher

Tomasz Kubik

Academic year 2025/2026 – April 12, 2026

Contents

Introduction **2**

1 Database structure **2**

1.1 General Overview . . . 2
1.2 Table `events` . . . 3
1.3 Table `organizations` . . . 4
1.4 Table `users` . . . 4
1.5 Table `event_registrations` . . . 4
1.6 Table `event_creations` . . . 5

2 Implementation **6**

3 Tests **10**

3.1 Running the application . . . 10
3.2 Registration of an user . . . 11
3.3 Organization Creation . . . 12
3.4 Event Creation . . . 13
3.5 Event Registration . . . 15
3.5.1 Simple . . . 15
3.5.2 Advanced . . . 17
3.6 Adding an organization to an existed event . . . 18

Appendixes **21**

List of Tables . . . 21
List of Figures . . . 21
List of source codes . . . 22

Introduction

This document presents the Data Storage Structure for the *WrocłArts* system for the third laboratory of "Information Systems Modelling".

First, we present the *database structure* 1, then some details about the implementation 2 and we expose finally our tests 3.

1 Database structure

1.1 General Overview

The Figure 1 presents the five implemented tables for the different entity : user, events and organizations and the connection-table such as the registration to event and creation of an event. This structure includes *many-to-many* relationship for the registration and the creation of an event.

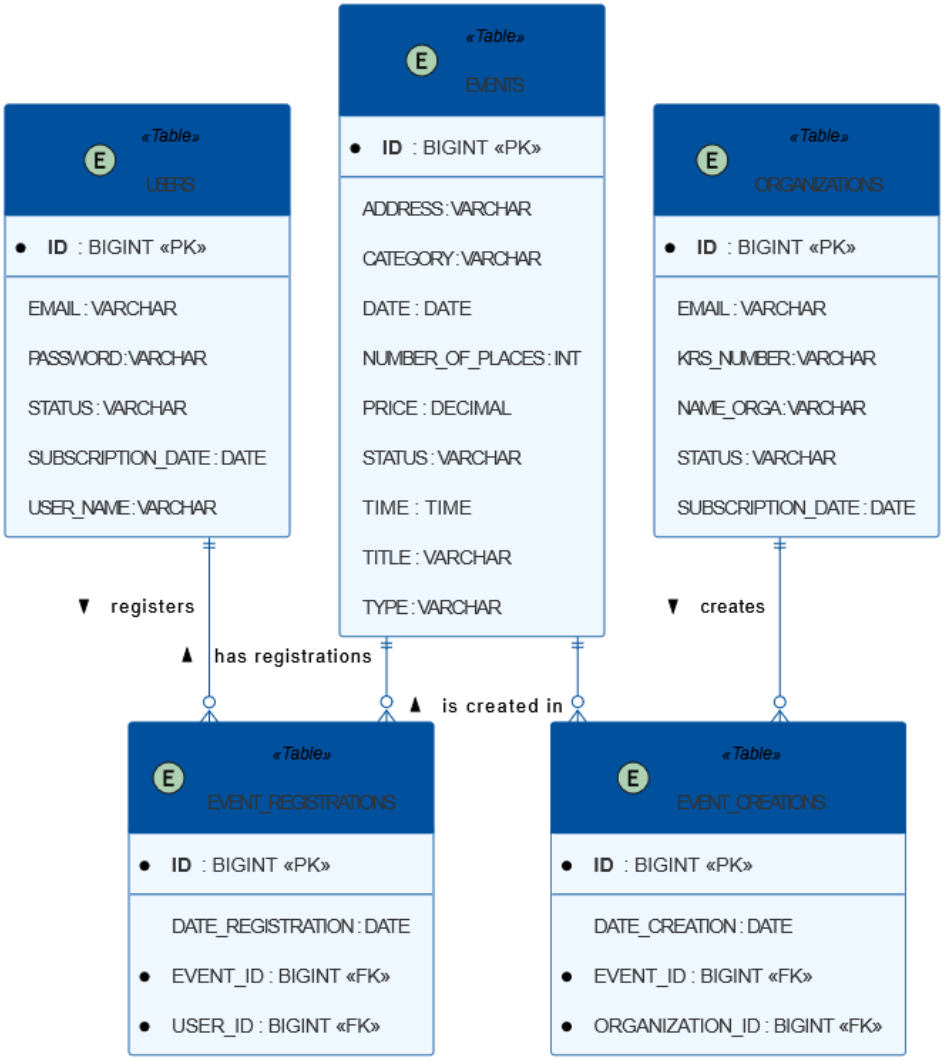

Figure 1: **Database structure.** We can see the relation *many-to-many* : each user can register to many events and many user cans register to a specific event. Each events is created by one or many organization and an organization can create many events.

1.2 Table events

id	address	category	date	numberOfPlaces	price	status	time	title	type
1	Nadodrze	Crafts	2026-10-14	15	120.0	Published	18:00:00	Pottery Workshop	In-Person
2	Śródmieście, Wrocław	Painting	2026-10-07	12	95.0	Published	17:00:00	Watercolor Painting Class	In-Person
3	Stare Miasto, Wrocław	Drawing	2026-10-10	20	60.0	Published	10:00:00	Urban Sketching Walk	In-Person
4	Nadodrze, Wrocław	Crafts	2026-10-15	10	140.0	Published	16:00:00	Ceramic Sculpture Workshop	In-Person
5	Krzyki, Wrocław	Music	2026-10-18	25	50.0	Published	19:00:00	Choir Singing for Beginners	In-Person
6	Muzeum Współczesne, pl. Wolności, Wrocław	Art Gallery	2026-10-21	18	30.0	Published	14:00:00	Contemporary Art Gallery Tour	In-Person
7	Rynek, Wrocław	Photography	2026-10-24	14	110.0	Published	09:00:00	Street Photography Workshop	In-Person
8	Przedmieście Świdnickie, Wrocław	Drawing	2026-10-28	10	85.0	Published	18:30:00	Portrait Drawing Masterclass	In-Person
9	Fabryczna, Wrocław	Music	2026-11-04	16	70.0	Published	18:00:00	Voice & Vocal Technique Workshop	In-Person
10	Biskupin, Wrocław	Painting	2026-11-08	12	100.0	Published	15:00:00	Acrylic Painting on Canvas	In-Person
11	Ołbin, Wrocław	Photography	2026-11-14	8	130.0	Published	17:30:00	Portrait Photography & Lighting	In-Person

Table 1: Description of the table `events` and fake data within to illustrate the structure. *The fake data were generate with an AI.*

1.3 Table organizations

id	email	krsNumber	nameOrga	status	subscriptionDate
1	contact@ceramicstudio.pl	0000123456	Ceramic Studio Wrocław	PendingVerification	2026-04-10
2	contact@artworkshop-wroclaw.pl	0000654321	ArtWorkshop Wrocław	PendingVerification	2026-04-11

Table 2: Description of the table `organizations` and fake data within to illustrate the structure.

1.4 Table users

id	email	password	status	subscriptionDate	userName
1	tomasz.kubik@pwr.edu.pl	psw	ACTIVE	2026-04-11	T_Kubik

Table 3: Description of the table `users` and fake data within to illustrate the structure.

1.5 Table event_registrations

id	dateRegistration	eventId	userId
1	2026-04-11	1	1
2	2026-04-11	9	1

Table 4: Description of the table `event_registrations` and fake data within to illustrate the structure.

1.6 Table `event_creations`

id	dateCreation	eventId	organizationId
1	2026-04-11	1	1
2	2026-04-11	2	1
3	2026-04-11	3	1
4	2026-04-11	4	1
5	2026-04-11	5	1
6	2026-04-11	6	1
7	2026-04-11	7	1
8	2026-04-11	8	1
9	2026-04-11	9	1
10	2026-04-11	10	1
11	2026-04-11	11	1
12	2026-04-11	8	2

Table 5: Description of the table `event_creations` and fake data within to illustrate the structure.

2 Implementation

The implemented code can be found here https://github.com/MaximeLuce/WroclArts.

We present one of the main Controller Listing 1 : the `EventsController`.

Listing 1: `EventsController.java` Example of Controller implemented.

```java
package pl.edu.pwr.tkubik.ism.api;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.RestController;
import pl.edu.pwr.tkubik.ism.model.EventEntity;
import pl.edu.pwr.tkubik.ism.service.EventRegistrationService;
import pl.edu.pwr.tkubik.ism.service.EventService;
import pl.edu.pwr.tkubik.ism.service.EventCreationService;
import pl.edu.pwr.tkubik.ism.service.OrganizationService;
import pl.edu.pwr.tkubik.ism.api.EventApi;
import pl.edu.pwr.tkubik.ism.model.Event;

import java.time.LocalDate;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;

@RestController
public class EventsController implements EventApi {

    @Autowired
    private EventService eventService;

    @Autowired
    private EventRegistrationService eventRegistrationService;

    @Autowired
    private EventCreationService eventCreationService;

    @Autowired
    private OrganizationService organizationService;

    // private method to convert an Entity (JPA) to a DTO (OpenAPI)
    private Event convertToDto(EventEntity entity) {
        Event dto = new Event();
        dto.setEventId(entity.getId());
        dto.setTitle(entity.getTitle());

        // convert LocalDate (DB) to String (DTO)
        if (entity.getDate() != null) {
            dto.setDate(entity.getDate().toString());
        }

        // Convert LocalTime (DB) to String (DTO)
        if (entity.getTime() != null) {
            dto.setTime(entity.getTime().toString());
        }

```

```
51          dto.setAddress(entity.getAddress());
52          dto.setCategory(entity.getCategory());
53          dto.setPrice(entity.getPrice());
54          dto.setNumberOfPlaces(entity.getNumberOfPlaces());
55          dto.setType(entity.getType());
56          dto.setStatus(entity.getStatus());
57
58          return dto;
59      }
60
61      // POST /events
62      @Override
63      public ResponseEntity<Event> addEvent(Event eventDto) {
64          // database Entity instance
65          EventEntity entity = new EventEntity();
66
67          // mappin DTO to Entity
68          entity.setTitle(eventDto.getTitle());
69
70          if (eventDto.getDate() != null) {
71              entity.setDate(LocalDate.parse(eventDto.getDate()));
72          }
73
74          if (eventDto.getTime() != null) {
75              entity.setTime(LocalTime.parse(eventDto.getTime()));
76          }
77
78          entity.setAddress(eventDto.getAddress());
79          entity.setCategory(eventDto.getCategory());
80          entity.setPrice(eventDto.getPrice());
81          entity.setNumberOfPlaces(eventDto.getNumberOfPlaces());
82          entity.setType(eventDto.getType());
83          entity.setStatus(eventDto.getStatus());
84
85          // call the service to save the event in the DB
86          EventEntity savedEntity = eventService.addEvent(entity);
87
88          // creating the link in the even_creations table
89
90          // we suppose by default that organization with ID=1 create this event
91          pl.edu.pwr.tkubik.ism.model.OrganizationEntity dummyOrg = new pl.edu.pwr.
                tkubik.ism.model.OrganizationEntity();
92          dummyOrg.setId(1L);
93
94          // creating the link
95          pl.edu.pwr.tkubik.ism.model.EventCreation creation = new pl.edu.pwr.tkubik.
                ism.model.EventCreation();
96          creation.setEvent(savedEntity);
97          creation.setOrganization(dummyOrg);
98          creation.setDateCreation(LocalDate.now());
99
100         // save the link in DB
101         eventCreationService.addCreation(creation);
102
103         // retrieve the ID generated by the DB and set it in the DTO
104         eventDto.setEventId(savedEntity.getId());
105
106         // return the final DTO with the 200 HTTP status
```

```
107         return new ResponseEntity<>(eventDto, HttpStatus.OK);
108     }
109
110     // POST /events/{eventId}/organizations/{orgId}
111     @Override
112     public ResponseEntity<Event> addOrganizationToEvent(Long eventId, Long orgId) {
113
114         // check if the event exists
115         EventEntity event = eventService.findEventById(eventId);
116         if (event == null) {
117             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
118         }
119
120         // check if the organization exists
121         pl.edu.pwr.tkubik.ism.model.OrganizationEntity organization =
                organizationService.findOrganizationById(orgId);
122         if (organization == null) {
123             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
124         }
125
126         // create the new link in the table "event_creations"
127         pl.edu.pwr.tkubik.ism.model.EventCreation creation = new pl.edu.pwr.tkubik.
                ism.model.EventCreation();
128         creation.setEvent(event);
129         creation.setOrganization(organization);
130         creation.setDateCreation(LocalDate.now());
131
132         // save in DB
133         eventCreationService.addCreation(creation);
134
135         // return the event to confirm
136         return new ResponseEntity<>(convertToDto(event), HttpStatus.OK);
137     }
138
139     // GET /events
140     @Override
141     public ResponseEntity<List<Event>> findAllEvents(Double lat, Double lng, Integer
            radius, String category, String keyword, LocalDate date) {
142         List<EventEntity> entities = eventService.findAllEvents();
143
144         List<Event> dtos = new ArrayList<>();
145         entities.forEach(entity -> dtos.add(convertToDto(entity)));
146
147         return new ResponseEntity<>(dtos, HttpStatus.OK);
148     }
149
150     // GET /events/{eventId}
151     @Override
152     public ResponseEntity<Event> findEventById(Long eventId) {
153         EventEntity entity = eventService.findEventById(eventId);
154
155         if (entity == null) {
156             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
157         }
158
159         return new ResponseEntity<>(convertToDto(entity), HttpStatus.OK);
160     }
161
```

```
162     // PUT /events
163     @Override
164     public ResponseEntity<Event> updateEvent(Event eventDto) {
165         if (eventDto.getEventId() == null) {
166             return new ResponseEntity<>(HttpStatus.BAD_REQUEST);
167         }
168 
169         EventEntity existingEntity = eventService.findEventById(eventDto.getEventId()
                );
170 
171         if (existingEntity == null) {
172             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
173         }
174 
175         existingEntity.setTitle(eventDto.getTitle());
176 
177         if (eventDto.getDate() != null) {
178             existingEntity.setDate(LocalDate.parse(eventDto.getDate()));
179         }
180 
181         // Convert String (DTO) to LocalTime (DB)
182         if (eventDto.getTime() != null) {
183             existingEntity.setTime(LocalTime.parse(eventDto.getTime()));
184         }
185 
186         existingEntity.setAddress(eventDto.getAddress());
187         existingEntity.setCategory(eventDto.getCategory());
188         existingEntity.setPrice(eventDto.getPrice());
189         existingEntity.setNumberOfPlaces(eventDto.getNumberOfPlaces());
190         existingEntity.setType(eventDto.getType());
191         existingEntity.setStatus(eventDto.getStatus());
192 
193         EventEntity updatedEntity = eventService.addEvent(existingEntity);
194 
195         return new ResponseEntity<>(convertToDto(updatedEntity), HttpStatus.OK);
196     }
197 
198     // DELETE /events/{eventId}
199     @Override
200     public ResponseEntity<Event> deleteEventById(Long eventId) {
201         EventEntity existingEntity = eventService.findEventById(eventId);
202 
203         if (existingEntity == null) {
204             return new ResponseEntity<>(HttpStatus.NOT_FOUND);
205         }
206 
207         eventService.deleteEventById(eventId);
208 
209         return new ResponseEntity<>(convertToDto(existingEntity), HttpStatus.OK);
210     }
211 
212     // POST /events/{eventId}/tickets
213     @Override
214     public ResponseEntity<Event> registerForEvent(Long eventId, String body) {
215         // check if the event exists
216         EventEntity event = eventService.findEventById(eventId);
217 
218         if (event == null) {
```

```
219             return new ResponseEntity<>(HttpStatus.NOT_FOUND); // Erreur 404
220         }
221
222         // we suppose by default that we use the user with ID=1
223         pl.edu.pwr.tkubik.ism.model.UserEntity dummyUser = new pl.edu.pwr.tkubik.ism.
                model.UserEntity();
224         dummyUser.setId(1L);
225
226         // create the link
227         pl.edu.pwr.tkubik.ism.model.EventRegistration registration = new pl.edu.pwr.
                tkubik.ism.model.EventRegistration();
228         registration.setEvent(event);
229         registration.setUser(dummyUser);
230         registration.setDateRegistration(LocalDate.now());
231
232         // save the link in the table "event_registrations"
233         eventRegistrationService.addRegistration(registration);
234
235         // we return the event to confirm and the HTTPO status ok
236         return new ResponseEntity<>(convertToDto(event), HttpStatus.OK);
237     }
238
239     // PUT /tickets/{ticketId}/scan
240     @Override
241     public ResponseEntity<Event> scanTicket(String ticketId, String body) {
242         return new ResponseEntity<>(HttpStatus.NOT_IMPLEMENTED);
243     }
244 }
```

3 Tests

3.1 Running the application

Once the project loaded, you need to proceed as follows:

- Run the paven project with `mvn clean compile`
- Run the application with our IDE
- Test the different routes available at http://localhost:8080/WroclArts/swagger-ui/index.html
- The updated data in the database can be seen at http://localhost:8080/WroclArts/h2-ui/
  - username: `sa`
  - password: (none)

3.2 Registration of an user

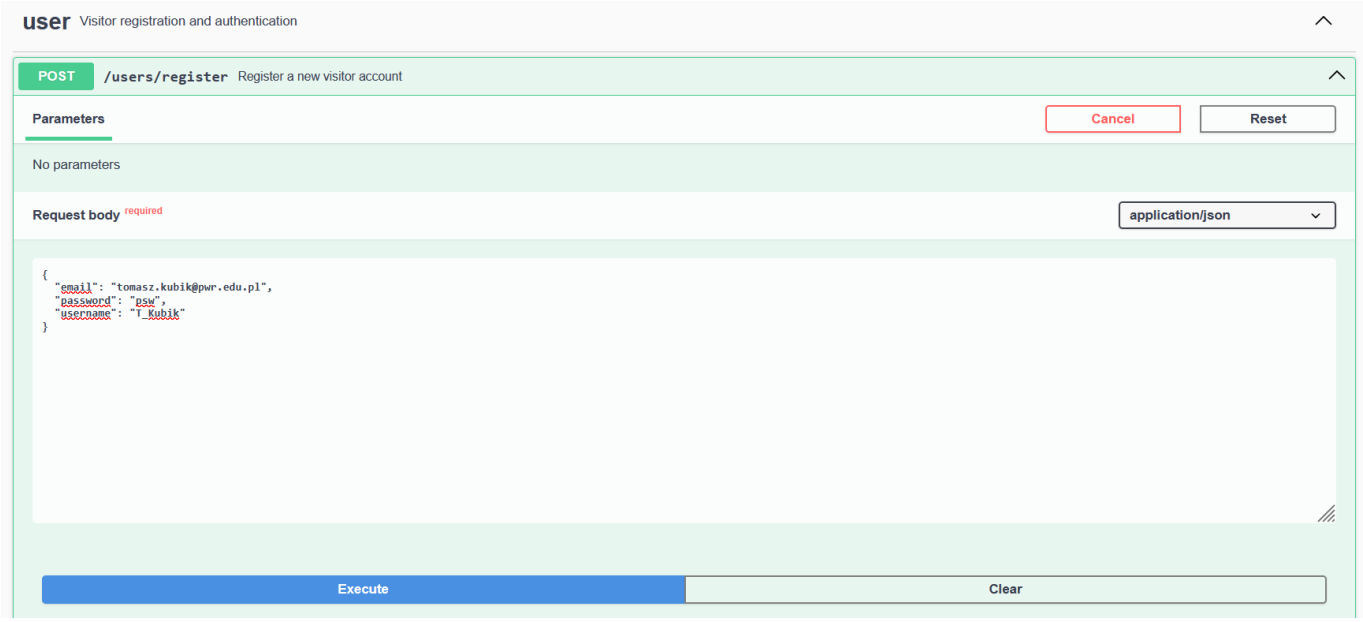

Figure 2: **Request POST for the creation of an user.** The route is `/users/register` and the request body contains the required JSON for the entity "User".

Figure 3: **Response of the *backend* for the creation of an user.** The response contains the status (200, OK) and the JSON of the inserted data of the new entity "User".

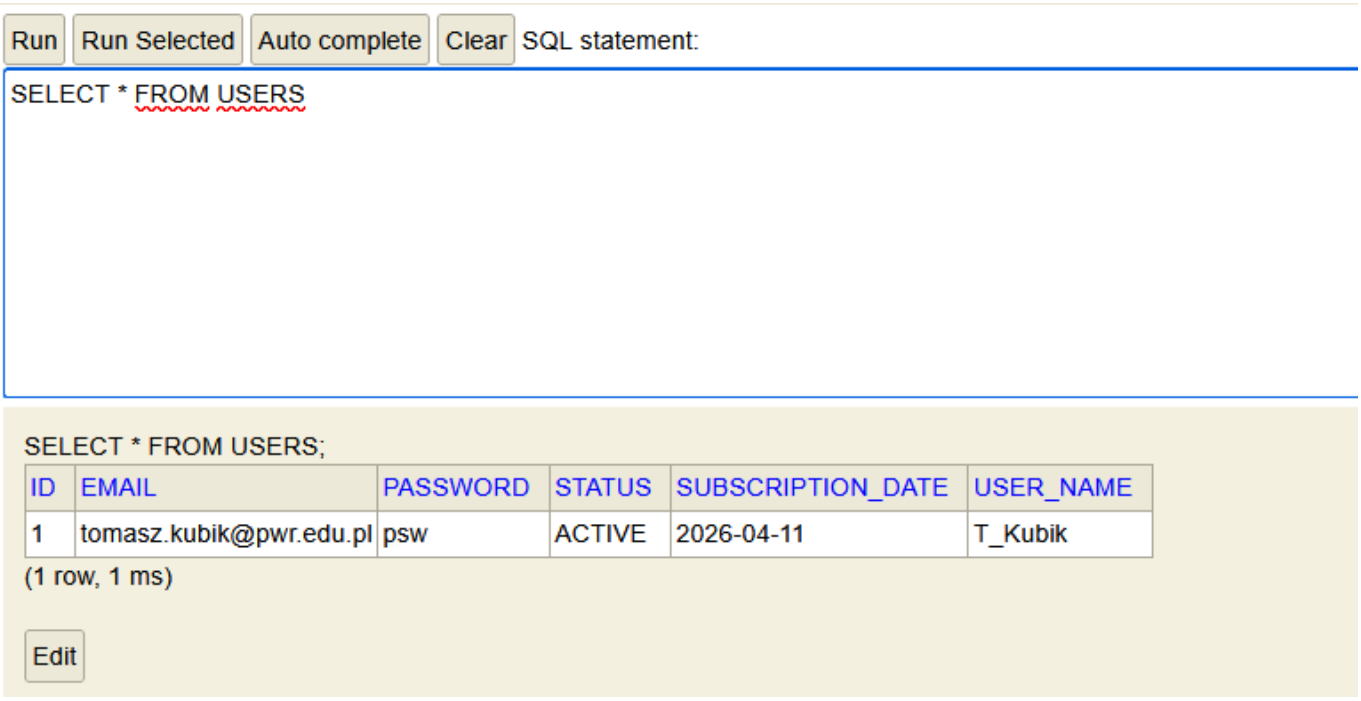

Figure 4: **Inserted data into the database, screenshot of H2 Console.** The new user is registered in the table `users`.

3.3 Organization Creation

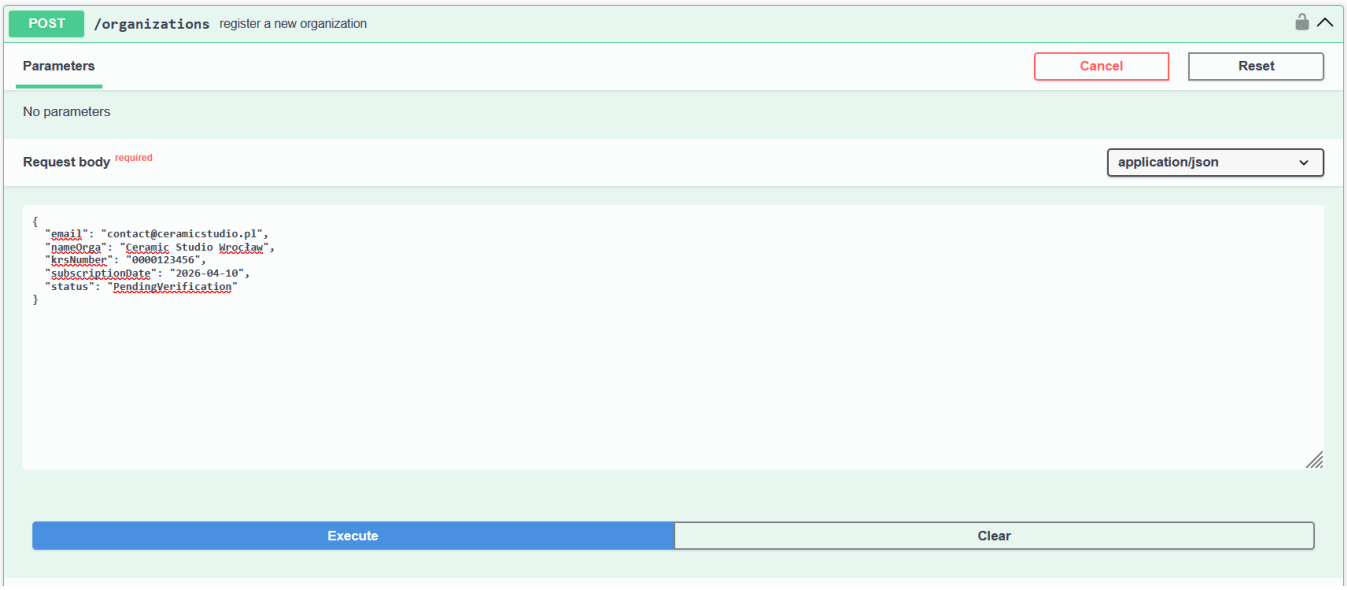

Figure 5: **Request POST for the creation of an user.** The route is `/organizations` and the request body contains the required JSON for the entity "Organization".

Figure 6: **Response of the *backend* for the creation of an organization.** The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Organization".

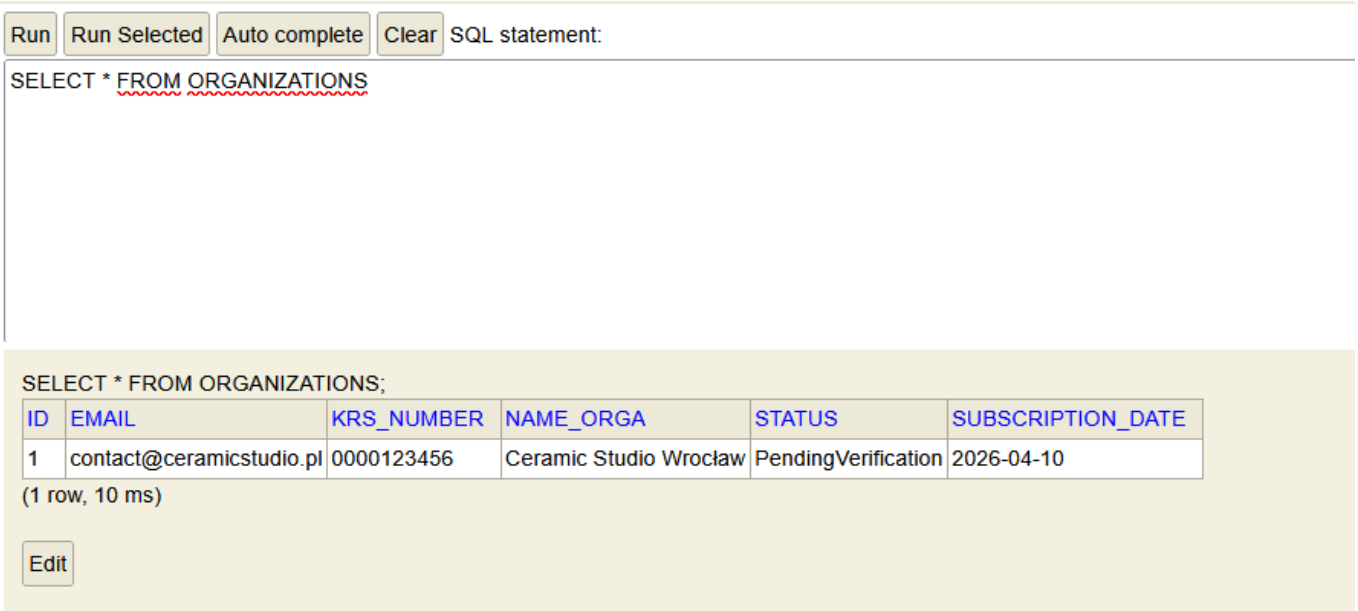

Figure 7: **Inserted data into the database, screenshot of H2 Console.** The new user is registered in the table `organizations`.

3.4 Event Creation

We suppose that the default organization that creates the event has the `id=1`.

// Simulation of the Organization (ID 1) creating this event pl.edu.pwr.tkubik.ism.model.OrganizationEntity dummyOrg = new pl.edu.pwr.tkubik.ism.model.OrganizationEntity(); dummyOrg.setId(1L); // Assumes Organization with ID 1 exists

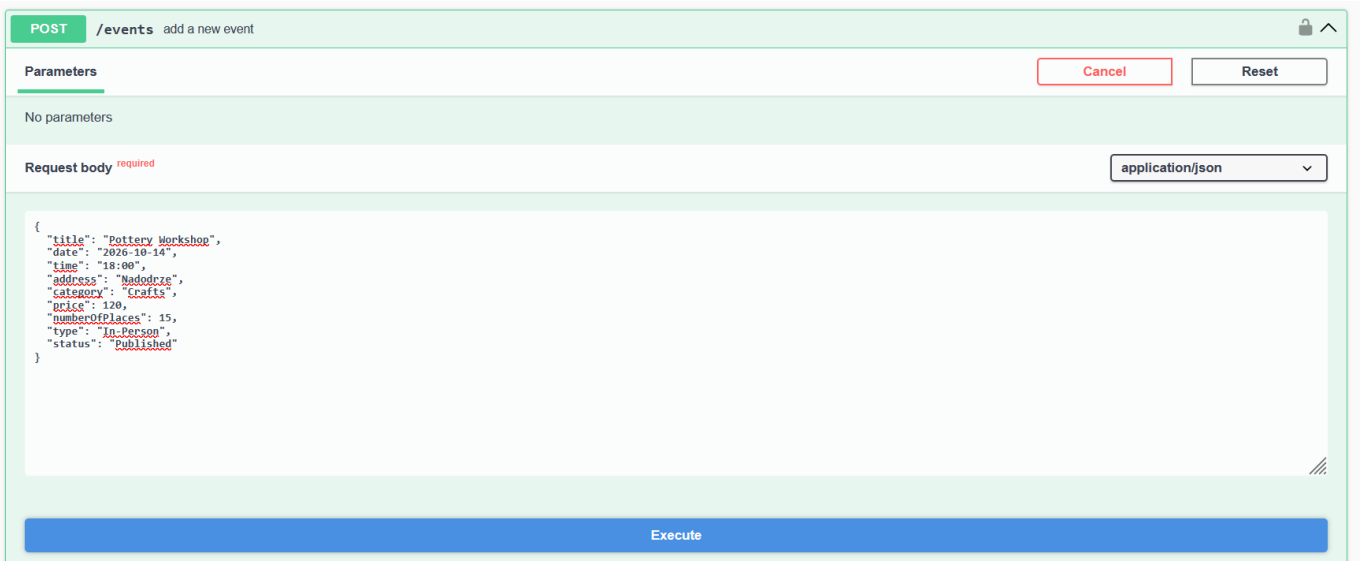

Figure 8: **Request POST for the creation of an user.** The route is `/events` and the request body contains the required JSON for the entity "Event".

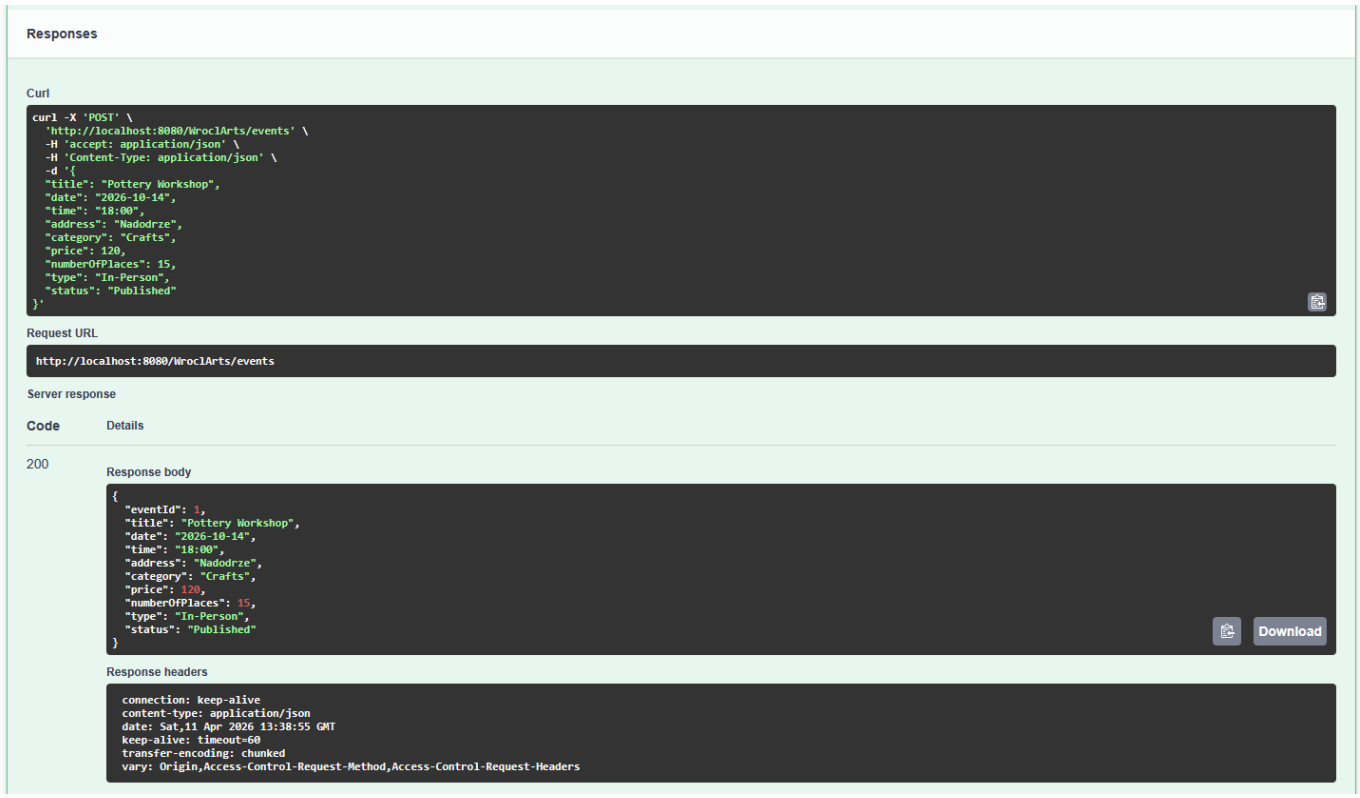

Figure 9: **Response of the *backend* for the registration to an event.** The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Event".

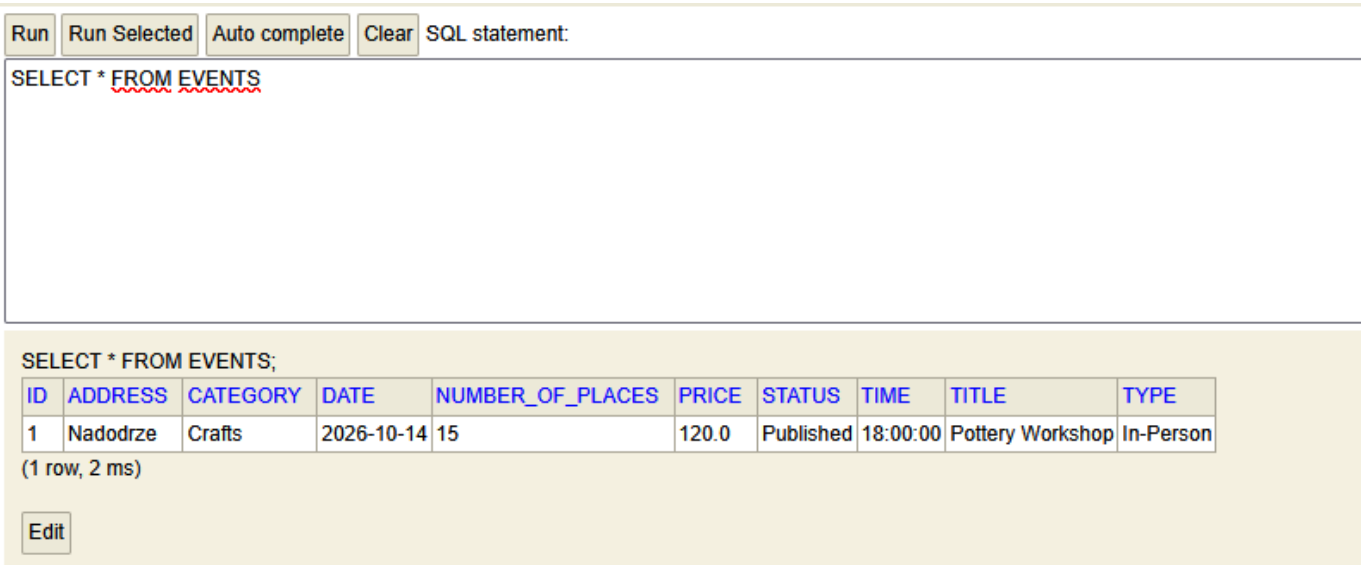

Figure 10: **Inserted data into the database, screenshot of H2 Console.** The new user is registered in the table `events`.

3.5 Event Registration

3.5.1 Simple

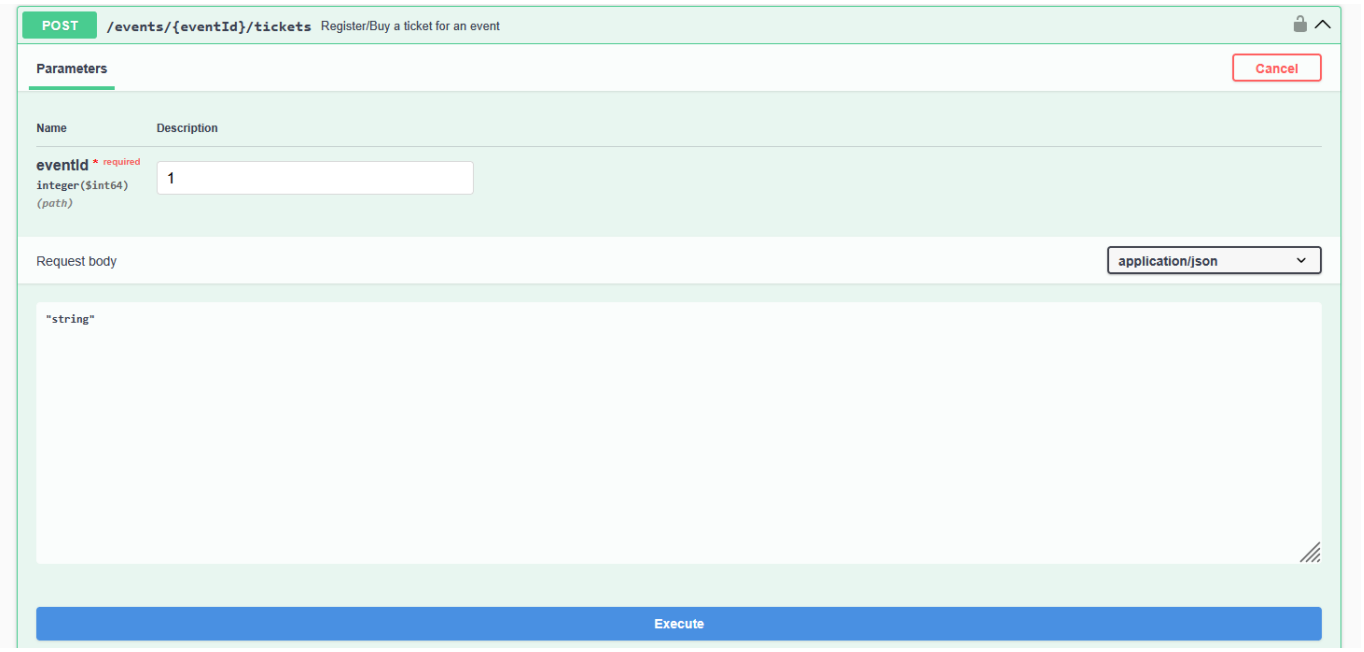

Figure 11: **Request POST for the registration of an event.** The route is `/events/{eventId}/tickets` and the request body contains additional string data to qualify the event (not implemented yet).

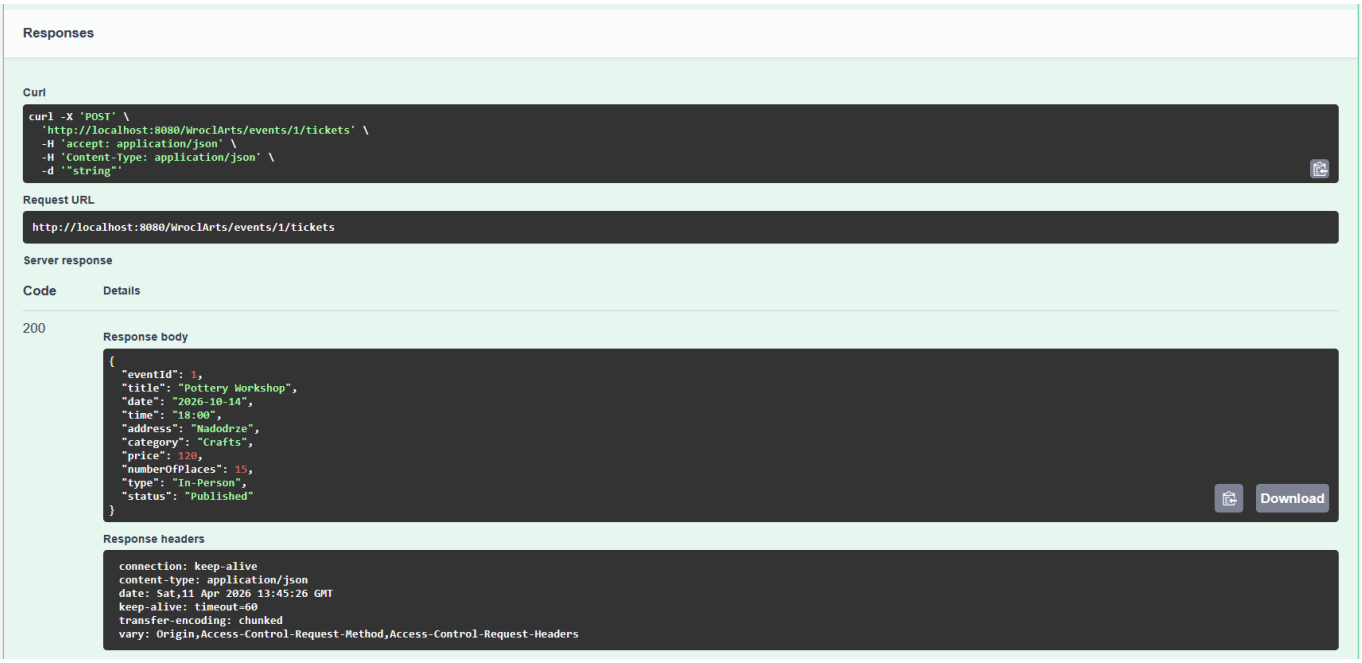

Figure 12: **Response of the *backend* for the registration to an event.** The response contains the status (200, OK) and the JSON of the data of the selected "Event".

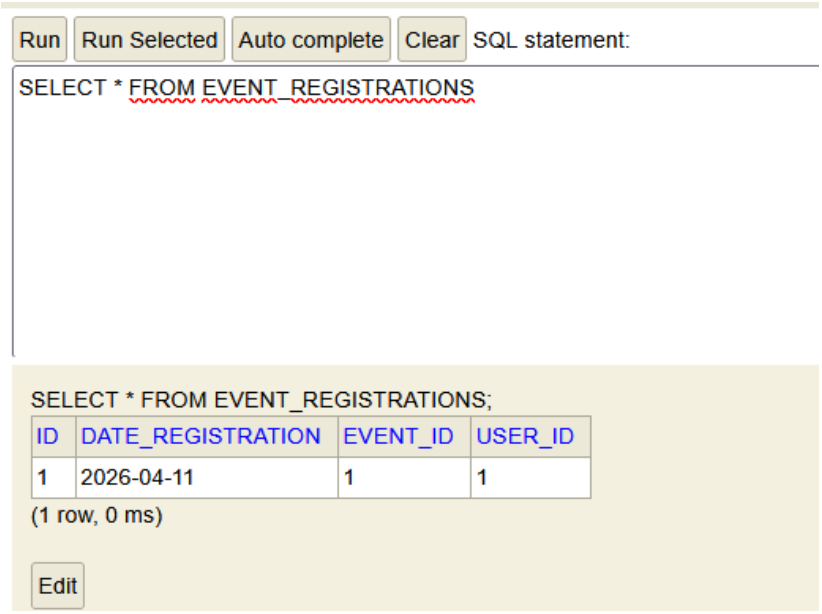

Figure 13: **Inserted data into the database, screenshot of H2 Console.** The new registration is registered in the table `event_registrations`.

3.5.2 Advanced

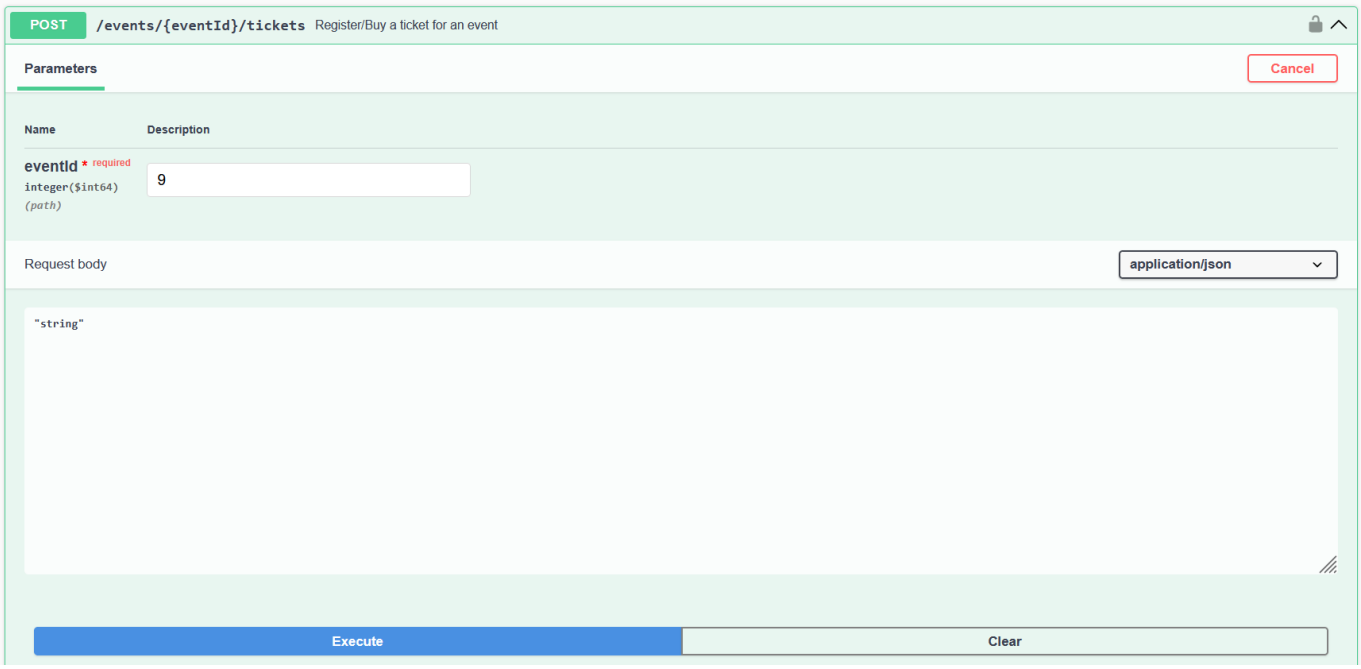

Figure 14: **Request POST for the registration of an event in an advanced case.** The route is `/events/{eventId}/tickets` and the request body contains additional string data to qualify the event (not implemented yet).

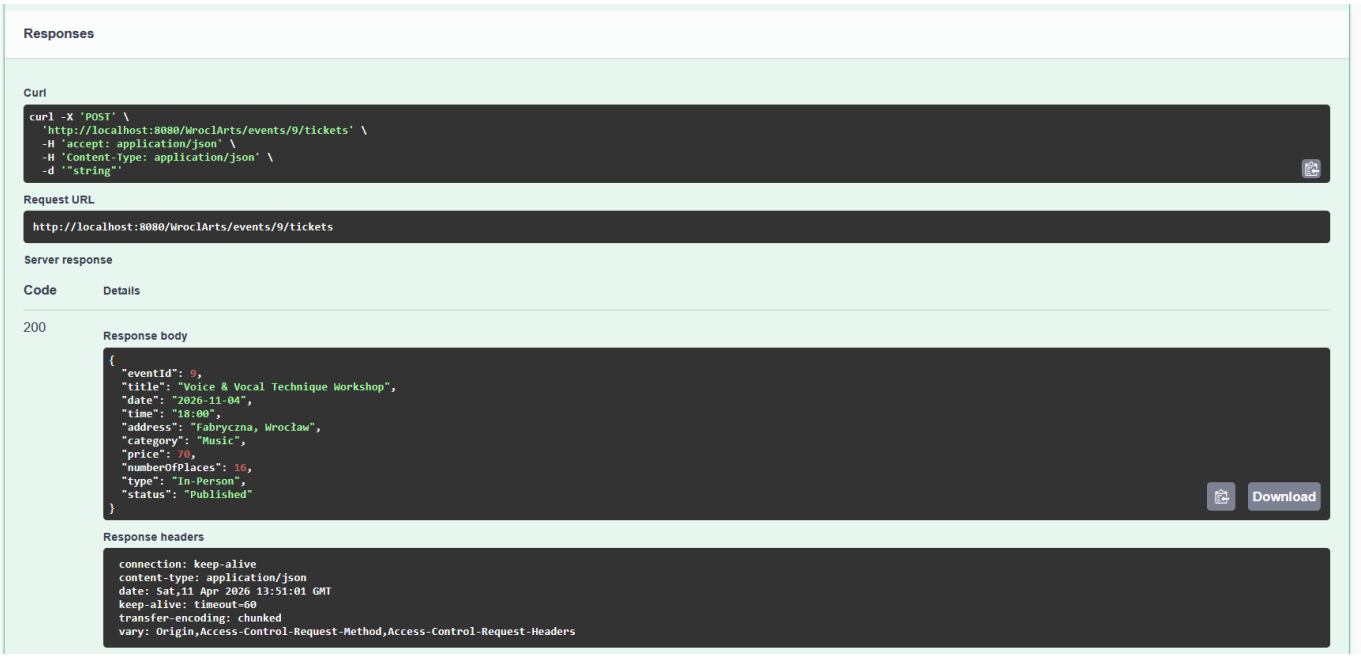

Figure 15: **Response of the *backend* for the creation of an event in an advanced case.** The response contains the status (200, OK) and the JSON of the data of the selected "Event".

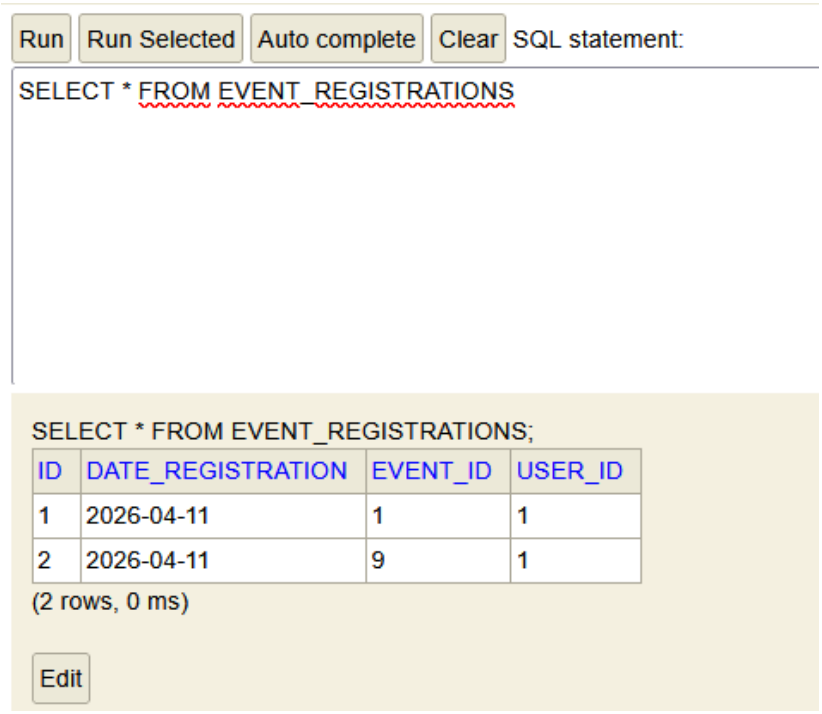

Figure 16: **Inserted data into the database, screenshot of H2 Console.** The new registration in an advanced case is registered in the table `event_registrations`.

3.6 Adding an organization to an existed event

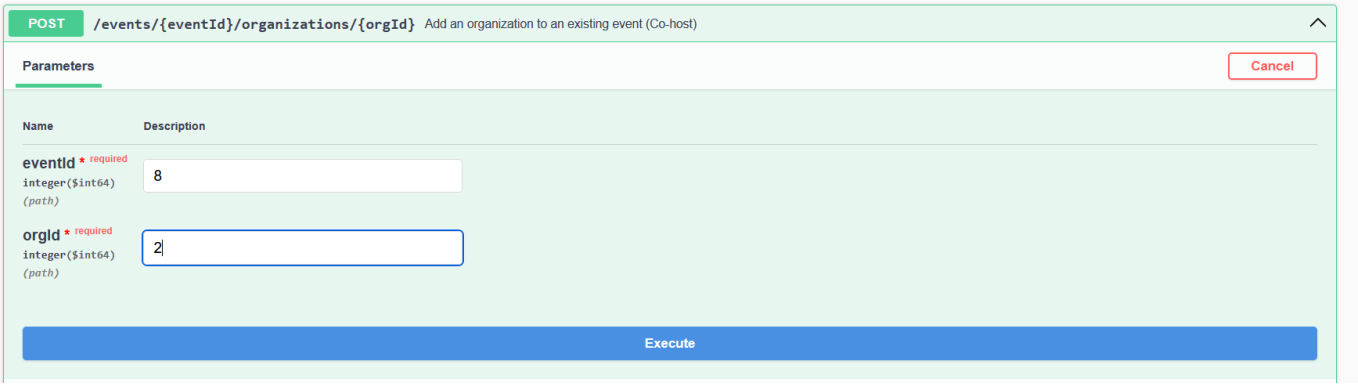

Figure 17: **Request POST for the adding of an organization to an existing event.** The route is `/events/{eventId}/organizations/{orgId}` and the request body contains the `eventid` and the `orgId`.

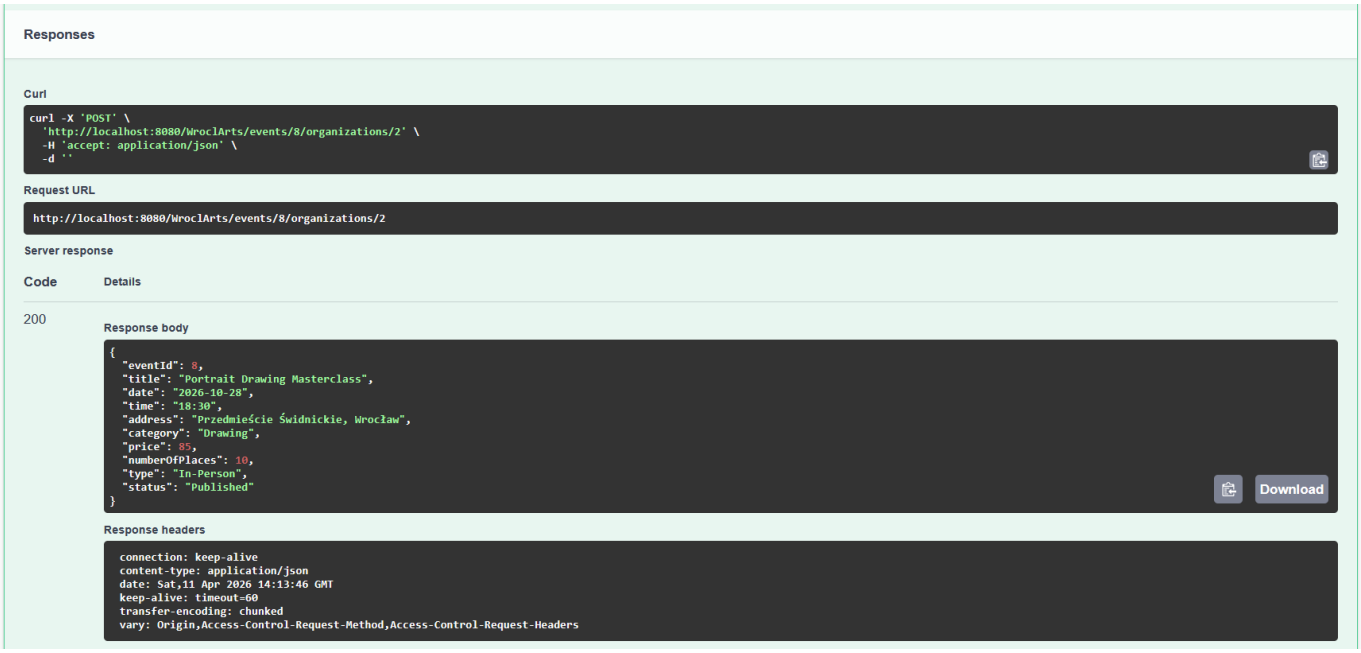

Figure 18: **Response of the *backend* for the adding of an organization to an existing event.** The response contains the status (200, OK) and the JSON of the data of the selected "Event".

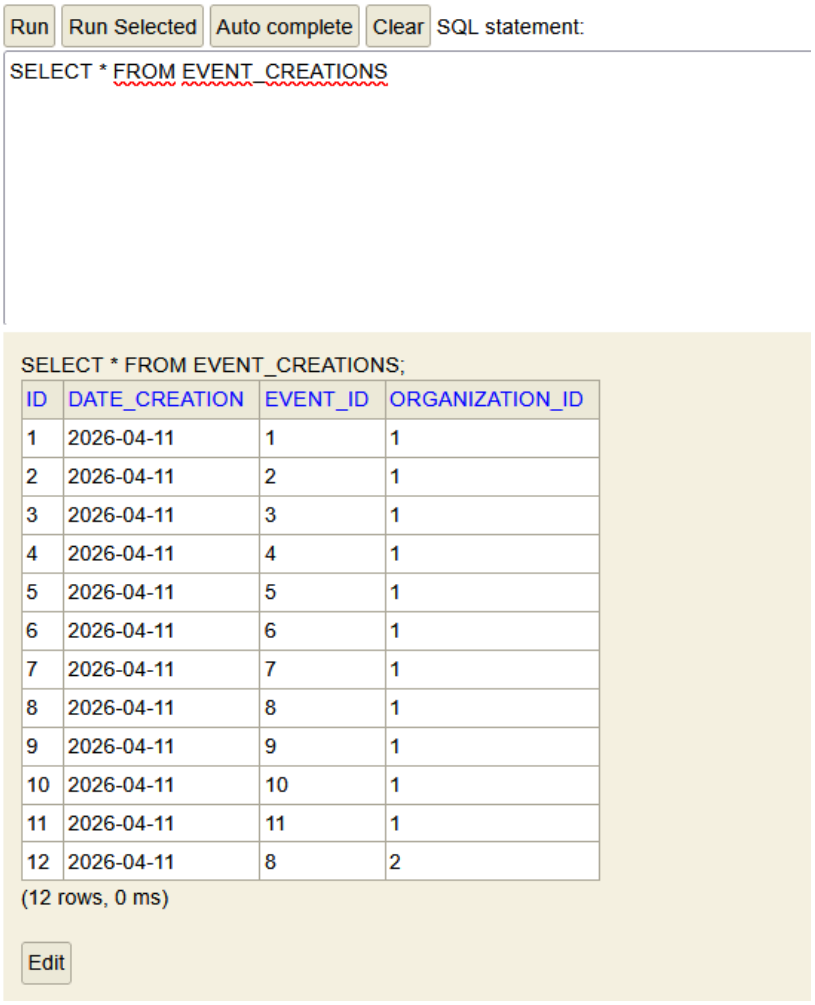

Figure 19: **Inserted data into the database, screenshot of H2 Console.** The new organization is registered in the table `event_creations`.

Appendixes

List of Tables

1 Description of the table `events` and fake data within to illustrate the structure. *The fake data were generate with an AI.* . . . 3
2 Description of the table `organizations` and fake data within to illustrate the structure. 4
3 Description of the table `users` and fake data within to illustrate the structure. . . . 4
4 Description of the table `event_registrations` and fake data within to illustrate the structure. . . . 4
5 Description of the table `event_creations` and fake data within to illustrate the structure. . . . 5

List of Figures

1 **Database structure.** We can see the relation *many-to-many* : each user can register to many events and many user cans register to a specific event. Each events is created by one or many organization and an organization can create many events. . . . 2
2 **Request POST for the creation of an user.** The route is `/users/register` and the request body contains the required JSON for the entity "User". . . . 11
3 **Response of the *backend* for the creation of an user.** The response contains the status (200, OK) and the JSON of the inserted data of the new entity "User". . . . 11
4 **Inserted data into the database, screenshot of H2 Console.** The new user is registered in the table `users`. . . . 12
5 **Request POST for the creation of an user.** The route is `/organizations` and the request body contains the required JSON for the entity "Organization". . . . 12
6 **Response of the *backend* for the creation of an organization.** The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Organization". . . . 13
7 **Inserted data into the database, screenshot of H2 Console.** The new user is registered in the table `organizations`. . . . 13
8 **Request POST for the creation of an user.** The route is `/events` and the request body contains the required JSON for the entity "Event". . . . 14
9 **Response of the *backend* for the registration to an event.** The response contains the status (200, OK) and the JSON of the inserted data of the new entity "Event". . . . 14
10 **Inserted data into the database, screenshot of H2 Console.** The new user is registered in the table `events`. . . . 15
11 **Request POST for the registration of an event.** The route is `/events/{eventId}/tickets` and the request body contains additional string data to qualify the event (not implemented yet). . . . 15
12 **Response of the *backend* for the registration to an event.** The response contains the status (200, OK) and the JSON of the data of the selected "Event". . . . 16

13 **Inserted data into the database, screenshot of H2 Console.** The new registration is registered in the table `event_registrations`. 16

14 **Request POST for the registration of an event in an advanced case.** The route is `/events/{eventId}/tickets` and the request body contains additional string data to qualify the event (not implemented yet). 17

15 **Response of the *backend* for the creation of an event in an advanced case.** The response contains the status (200, OK) and the JSON of the data of the selected "Event". . . 17

16 **Inserted data into the database, screenshot of H2 Console.** The new registration in an advanced case is registered in the table `event_registrations`. 18

17 **Request POST for the adding of an organization to an existing event.** The route is `/events/{eventId}/organizations/{orgId}` and the request body contains the `eventid` and the `orgId`. 18

18 **Response of the *backend* for the adding of an organization to an existing event.** The response contains the status (200, OK) and the JSON of the data of the selected "Event". 19

19 **Inserted data into the database, screenshot of H2 Console.** The new organization is registered in the table `event_creations`. 20

List of source codes

1 `EventsController.java` Example of Controller implemented. 6